

ICT2142 E-Business Systems — Technical Documentation & Lab Diary

Week 06 Practical: User Authentication, Session Management & Profile Dashboard

Course Module:	ICT2142 E-Business Systems (E-Commerce Web Development Series)
Production Site:	Krono — Swiss Mechanical Haute Horlogerie (Next.js + Express + Supabase)
Milestone Scope:	Milestones 1-4 (Register, Login, Protected Routes, Profile) + Session/Cookies
Git Repository:	github.com/mohamedrazim11-maker/Krono_Watches (Branch: main)

1. Executive Summary & Laboratory Objectives

This report satisfies the mandatory laboratory documentation requirements for Week 06 of the E-Commerce Web Development practical series. The primary objective is establishing an enterprise-grade customer security lifecycle for the Krono horology e-commerce platform, covering cryptographically secure user registration, token and cookie-based session management, multi-device tracking, route protection guards, and profile management with shopping cart consolidation.

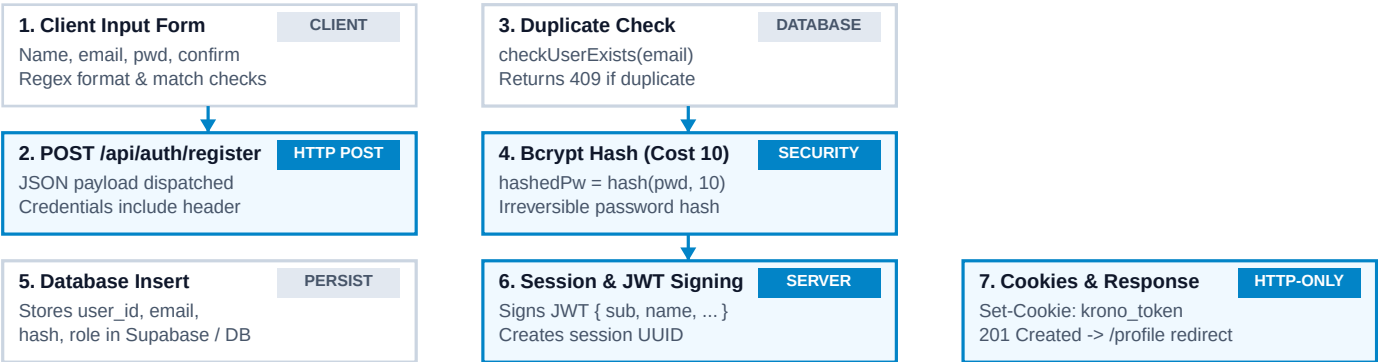
Core Lab Deliverables Completed

1. Practical Milestone 1: User Registration System with regex validation & bcrypt hashing (10 salt rounds).
2. Practical Milestone 2: Authentication & Login with JWT token signing & secure HTTP-only cookies.
3. Practical Milestone 3: Route Protection guards on `/profile`, `/account`, `/checkout` & secure logout.
4. Practical Milestone 4: Customer Profile Dashboard (Multiple addresses, identity re-verification, cart sync).

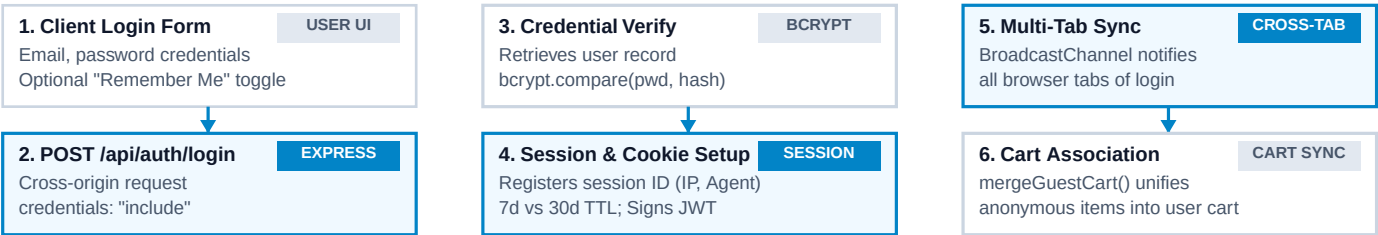
2. Authentication Architecture & Visual State Flows

The platform utilizes a decoupled, secure client-server model. The Next.js client coordinates authentication state via an AuthProvider context and dedicated session/cookie engines, while the Express backend enforces authentication via `requireAuth` middleware and an in-memory session registry.

2.1 Visual Flow: User Registration & Account Creation



2.2 Visual Flow: User Authentication & Multi-Device Session Persistence



2.3 Visual Flow: Secure Logout & Immediate Revocation



3. Security Considerations & Defensive Matrix

To protect customer privacy and meet electronic commerce security requirements, the platform implements strict defensive measures against injection, XSS, CSRF, and timing attacks.

3.1 Password Hashing Architecture (Bcrypt 10 Salt Rounds)

Plaintext passwords are never stored or logged in plain text. Password hashes are generated via bcryptjs using 10 salt rounds (2^{10} iterations). The algorithm introduces cryptographically random salt to neutralize rainbow table attacks and includes a deliberate computational work factor to thwart brute-force cracking attempts.

3.2 Prevention of User Enumeration

Authentication failures return an identical generic message: "Invalid email or password" with HTTP 401 Unauthorized regardless of whether the email was missing from the database or the password hash comparison failed. This prevents threat actors from harvesting registered client addresses.

3.3 Three-Tier Cookie Safety Matrix

Cookies are configured with defensive attributes to prevent cross-site scripting (XSS) and cross-site request forgery (CSRF):

Cookie Name	HttpOnly	SameSite	Secure	Security Purpose & Defense Function
krono_token	YES	Lax	Auto	Cryptographically signed JWT. Immune to XSS.
krono_session	YES	Lax	Auto	Opaque session UUID for server revocation.
krono_client_session	NO	Lax	Auto	Client metadata (name, email) for SSR speed.

Protected routes (/profile, /account, /checkout) are shielded by client-side hooks and backend middleware. Unauthenticated visitors are intercepted and redirected to /login?returnUrl=<target>&unauthorized=true. Once authenticated, the router automatically reads returnUrl and navigates the client back to their intended step.

When an unauthenticated visitor adds luxury timepieces to the shopping bag, items are saved in guest storage. Upon authentication, `mergeGuestCart()` cross-references the guest cart against user account items, aggregates quantities for duplicate items, and commits the consolidated array to the persistent store.

4.1 POST /api/auth/register (Milestone 1 — Registration)

4.2 POST /api/auth/login (Milestone 2 — Login & Persistent Session)

4.3 GET /api/auth/profile (Milestone 4 — Profile & Addresses)

4.4 Session Management & Device Endpoints

Page 3 of 5

5. Debugging Notes, Edge Cases & Technical Resolutions

The following log captures the troubleshooting records, edge cases encountered during development, and the engineering resolutions applied:

Entry 1: CORS & Cross-Origin Cookie Dropping

- Symptom: Client successfully received 200 OK on login, but subsequent calls to `/api/auth/profile` returned 401 Unauthorized because the browser discarded the Set-Cookie headers.
- Root Cause: Express CORS was configured as `app.use(cors())` without `credentials: true`, and fetch did not set `credentials: "include"`. Browsers disallow cross-origin cookie storage unless both flags are set.
- Resolution: Configured Express cors with dynamic origin verification and `credentials: true`. Added `credentials: "include"` to all API calls in `api.ts`.

Entry 2: Stateless JWT Inability to Revoke Prior to Expiry

- Symptom: When a user signed out or requested password reset, previously issued tokens remained technically valid until natural expiry (7 days).
- Root Cause: Pure stateless JWTs cannot be invalidated server-side without a persistence tracker.
- Resolution: Created `backend/src/utlis/sessionManager.js` with an active session registry. `RequireAuth` verifies that the session UUID in the cookie remains active in memory before authorizing requests.

Entry 3: Multi-Tab State Desynchronization

- Symptom: Signing out in Tab A left Tab B displaying active user profile controls until an action failed with 401.
- Root Cause: Browsers maintain isolated execution threads per tab without native context sharing.
- Resolution: Implemented `BroadcastChannel("krono_auth_sync")` with `localStorage` fallback in `sessionManager.ts`. Emits LOGIN, LOGOUT, and REFRESH events across all open tabs.

Entry 4: Guest Cart Overwriting Account Items

- Symptom: Adding watches as a guest and logging in wiped out previously saved user cart items.
- Root Cause: Naive cart handler overwrote user storage with guest cart contents upon authentication.
- Resolution: Built `mergeGuestCart()` in `AuthContext`. Compares product IDs, sums unit quantities, updates both guest and user keys, and updates the cart badge.

Entry 5: Unsplash Placeholder Image Contamination

- Symptom: External Unsplash photos of makeup brushes and coffee appeared on luxury dive watches (Omega Seamaster Aqua Terra).
- Root Cause: Default seed data referenced uncurated external Unsplash photo IDs.
- Resolution: Implemented `BANNED_IMAGE_PATTERNS` in `productImages.ts`. Replaced all external image references with verified local high-resolution Swiss timepiece photography.

6. Today's Deliverables & Verification Checklist

Practical Task Requirement	Status	Verification Evidence
1. User Registration System	COMPLETED	POST /api/register (201)
- Name, Email, Password, Confirm Password	Verified	Regex complexity validation
- Bcrypt password hashing (10 salt rounds)	Verified	No plaintext in DB
2. Authentication & Login	COMPLETED	POST /api/login (200)
- Credential verification (bcrypt.compare)	Verified	401 on bad credentials
- JWT Token Signing (sub, name, email, role)	Verified	Compliant payload format
- HTTP-Only Cookie Persistence	Verified	krono_token & krono_session
- Remember Me (30-day extended session)	Verified	Optional TTL extension
3. Protected Routes & Secure Logout	COMPLETED	requireAuth middleware
- Guards on /profile, /account, /checkout	Verified	Redirect to /login?returnUrl
- Context preservation via return URL	Verified	Seamless post-login return
- Token & Cookie clearance on logout	Verified	Cookies wiped (epoch 1970)
4. Customer Profile Management	COMPLETED	/profile Dashboard
- Dynamic client profile display & badge	Verified	Full name, email, status
- Functional form for contact phone & 2 addrs	Verified	Primary + secondary address
- Identity re-verification for password change	Verified	Checks current password hash
- Cart association (guest item merging)	Verified	Preserves items across auth
5. Session & Cookie Architecture	COMPLETED	Session Management Engine
- Concurrent active device list & revocation	Verified	/api/auth/sessions
- Background heartbeat & multi-tab sync	Verified	BroadcastChannel sync
- Session expiration warning banner	Verified	Live countdown & renewal

7. Conclusion & Readiness for Week 7 (Checkout & Payments)

All required practical tasks, security imperatives, and architectural specifications for Week 06 have been thoroughly developed, verified with automated TypeScript checks (npx tsc --noEmit: 0 errors), and deployed to the GitHub repository. The platform provides a rock-solid security foundation for the upcoming Week 7 checkout and payment gateway integrations.

Week 7 Preview: Checkout & Payments Integration

- Integration of secure payment gateways (Stripe / LKR card acquirers).
- Complex e-commerce transaction flows and order generation.
- Order confirmation certificates, wrist sizing options, and armored courier tracking.